

计算机设计实验：单周期、多周期与流水线

RISC-V CPU 设计报告

单位：_____ 姓名：_____ 学号：_____

2026 年 6 月

1 实验目的

本实验围绕 RV32I 指令子集 CPU 的实现、扩展和验收展开。实验中完成了单周期 CPU、多周期 CPU 和五级流水线 CPU 的设计，并在已有排序程序的基础上补充了最终验收要求覆盖的比较、逻辑立即数、移位立即数、条件分支以及跳转指令。单周期 CPU 用于验证指令数据通路和控制信号是否完整，多周期 CPU 用于观察指令在多个状态之间复用硬件资源的执行方式，流水线 CPU 用于在 IF、ID、EX、MEM、WB 五个阶段并行推进多条指令，并处理由并行执行带来的数据冒险和控制冒险。

最终验收关注两类证据。第一类证据是仿真结果，要求测试 1 和测试 3 跑通单周期 CPU，测试 2 和测试 4 跑通流水线 CPU，其中测试 1、测试 2 使用 30 条指令测试程序，测试 3、测试 4 使用学号排序程序。第二类证据是实现理解，要求能够结合代码解释一条运算指令、一条跳转或分支指令，以及流水线相对于单周期 CPU 增加的段寄存器、数据转发、暂停和冲刷机制。

2 实验环境与材料

实验在 WSL2 Linux 环境下完成 Verilog 仿真，使用 Icarus Verilog 11.0 编译 RTL，使用 vvp 运行仿真输出。测试材料来自教师给出的最终验收目录，其中 `expected_results.json` 记录了扩展指令测试点的期望寄存器结果，`sc_final_tb.v` 和 `pl_final_tb.v` 分别作为单周期和流水线 CPU 的验收 testbench。CPU 实现代码位于 `source-sc`、`source-mc` 和 `source-pipeline`，开发板相关封装位于 `nexys-a7`。

表 1: 实验环境

项目	内容
操作系统	Linux 6.6.87.2-microsoft-standard-WSL2, x86_64
Verilog 编译器	Icarus Verilog 11.0
仿真运行器	Icarus Verilog runtime 11.0
单周期仿真入口	source-sc/sccomp_tb.v 与 source-sc/sccomp.v
多周期仿真入口	source-mc/mccomp_tb.v 与 source-mc/mccomp.v
流水线仿真入口	source-pipeline/pipecomp_tb.v 与 source-pipeline/pipecomp.v
最终验收脚本	final-验收文档/run_final_tests.py

3 总体设计

单周期 CPU 以一条指令一个时钟周期为边界，SCCPU.v 在同一周期内完成指令字段拆分、寄存器堆读取、立即数扩展、ALU 运算、数据存储器访问、写回数据选择和下一条 PC 生成。该结构的优点是控制路径直观，指令行为容易从组合逻辑中追踪；代价是一个时钟周期必须容纳最慢指令的完整路径，因此周期长度由访存类或跳转类指令决定。

多周期 CPU 将一条指令拆分到取指、译码、执行、访存和写回等状态中完成。实验中使用 MCCPU.v 保存当前状态、指令寄存器和中间结果寄存器，使不同指令可以按需经过不同状态。该设计同一时刻仍只推进一条指令，因此不会产生流水线 CPU 中相邻指令重叠执行引起的 RAW 数据冒险或控制冒险，但需要通过状态机保证每一类指令在正确状态写 PC、写内存或写寄存器。

流水线 CPU 在 PipelineCPU.v 中维护 PC 和四组段寄存器，并将译码、ALU、分支判断和冒险控制拆成独立模块。拆分后的 pipe_decode.v 负责生成控制信号，pipe_alu.v 负责 EX 阶段算术逻辑运算，pipe_branch.v 负责条件分支和跳转目标计算，hazard_unit.v 负责数据转发、load-use 暂停和跳转冲刷。该结构使 IF、ID、EX、MEM、WB 五个阶段可以同时处理不同指令，但也要求显式处理指令之间的相关性。

表 2: 三类 CPU 设计差异

比较项	单周期 CPU	多周期 CPU	流水线 CPU
指令推进方式	一条指令在一个周期内完成	一条指令跨多个状态完成	多条指令处于不同阶段并行推进
中间状态保存	主要依赖组合路径	使用状态寄存器和中间结果寄存器	使用 IF/ID、ID/EX、EX/MEM、MEM/WB 段寄存器
控制重点	opcode 和 funct 字段直接产生控制信号	状态机决定当前周期执行动作	控制信号随段寄存器向后传递
冒险处理	不存在指令重叠导致的流水线冒险	同一时刻只处理一条指令, 不需要流水线冒险单元	需要转发、暂停和冲刷

4 单周期 CPU 的指令扩展实现

单周期 CPU 的控制器在 `ctrl.v` 中根据 `opcode`、`funct3` 和 `funct7` 生成 `RegWrite`、`MemWrite`、`EXTOp`、`ALUOp`、`NPCOp`、`ALUSrc` 和 `WDSe1`。R 型比较指令在 `opcode` 为 0110011 时译码, I 型立即数指令在 `opcode` 为 0010011 时译码, B 型分支在 `opcode` 为 1100011 时通过 `Funct3` 选择具体比较条件, `jal` 和 `jalr` 则分别选择 J 型或 I 型立即数并把写回来源切换到 `PC+4`。

Listing 1: 单周期控制器中扩展指令的译码方式

```

1 7'b0110011: begin // R-type
2     RegWrite = 1'b1;
3     case ({Funct7, Funct3})
4         {7'b0000000, 3'b000}: ALUOp = `ALUOp_add;
5         {7'b0100000, 3'b000}: ALUOp = `ALUOp_sub;
6         {7'b0000000, 3'b010}: ALUOp = `ALUOp_slt;
7         {7'b0000000, 3'b011}: ALUOp = `ALUOp_sltu;
8         {7'b0000000, 3'b100}: ALUOp = `ALUOp_xor;
9         {7'b0000000, 3'b110}: ALUOp = `ALUOp_or;
10        {7'b0000000, 3'b111}: ALUOp = `ALUOp_and;
11        {7'b0000000, 3'b001}: ALUOp = `ALUOp_sll;
12        {7'b0000000, 3'b101}: ALUOp = `ALUOp_srl;
13        {7'b0100000, 3'b101}: ALUOp = `ALUOp_sra;
14    default: begin

```

```

15     RegWrite = 1'b0;
16     ALUOp = `ALUOp_nop;
17     end
18     endcase
19 end

```

SCCPU.v 将 ALUSrc 作为 ALU 第二操作数选择信号。R 型指令的 ALUSrc 为 0，第二操作数来自寄存器堆读端口 RD2；I 型算术逻辑指令、访存指令和 jalr 的 ALUSrc 为 1，第二操作数来自 EXT.v 扩展后的立即数。写回阶段通过 WDSel 在 ALU 结果、数据存储器输出和 PC+4 之间选择，因而同一套写回端口可以覆盖普通运算、lw 以及 jal/jalr。

Listing 2: 单周期 CPU 中操作数选择、模块连接与写回选择

```

1  assign B = (ALUSrc) ? immout : RD2;
2  assign Data_out = RD2;
3
4  ctrl U_ctrl(
5      .Op(Op), .Funct7(Funct7), .Funct3(Funct3), .Zero(Zero),
6      .RegWrite(RegWrite), .MemWrite(mem_w),
7      .EXTOp(EXTOp), .ALUOp(ALUOp), .NPCOp(NPCOp),
8      .ALUSrc(ALUSrc), .WDSel(WDSel)
9  );
10
11 NPC U_NPC(.PC(PC_out), .NPCOp(NPCOp), .IMM(immout), .RS1(RD1), .NPC(NPC));
12 alu U_alu(.A(RD1), .B(B), .ALUOp(ALUOp), .C(aluout), .Zero(Zero));
13
14 always @* begin
15     case(WDSel)
16         `WDSel_FromALU: WD <= aluout;
17         `WDSel_FromMEM: WD <= Data_in;
18         `WDSel_FromPC:  WD <= PC_out + 4;
19         default: WD <= aluout;
20     endcase
21 end

```

ALU 的扩展重点是同时支持普通算术逻辑结果和分支条件结果。slt 和 slti 使用有符号比较，sltu 和 sltiu 使用无符号比较，移位指令只取第二操作数低 5 位作为移位量。分支类 ALUOp 直接产生 0 或 1 条件结果，Zero 对这些分支类操作解释为 $C \neq 0$ ，而对 beq/sub 类操作解释为 $C == 0$ 。这样控制器可以统一使用 Zero 决定 NPCOp 是否切换到分支目标。

Listing 3: 单周期 ALU 中比较、分支和移位的实现

```

1  `ALUOp_bne:  C = (A != B);
2  `ALUOp_blt:  C = (A < B);
3  `ALUOp_bge:  C = (A >= B);
4  `ALUOp_bltu: C = ($unsigned(A) < $unsigned(B));
5  `ALUOp_bgeu: C = ($unsigned(A) >= $unsigned(B));
6  `ALUOp_slt:  C = (A < B) ? 32'd1 : 32'd0;
7  `ALUOp_sltu: C = ($unsigned(A) < $unsigned(B)) ? 32'd1 : 32'd0;
8  `ALUOp_sll:  C = A << B[4:0];
9  `ALUOp_srl:  C = $unsigned(A) >> B[4:0];
10 `ALUOp_sra:  C = A >>> B[4:0];
11
12 assign Zero = (ALUOp == `ALUOp_bne ||
13                ALUOp == `ALUOp_blt ||
14                ALUOp == `ALUOp_bge ||
15                ALUOp == `ALUOp_bltu ||
16                ALUOp == `ALUOp_bgeu) ? (C != 32'b0) : (C == 32'b0);

```

5 流水线 CPU 的实现与冒险处理

流水线 CPU 与单周期 CPU 的根本差别在于指令是否重叠执行。单周期 CPU 在 `SCCPU.v` 中把取指、译码、执行、访存和写回放在同一个周期内完成，因此一条指令结束后才会进入下一条指令，不需要保存“前一阶段交给后一阶段”的信息。流水线 CPU 同一时刻可能有多条指令分别处于 IF、ID、EX、MEM 和 WB 阶段，因此 `PipelineCPU.v` 必须额外定义段寄存器，把每一级产生的数据和控制信号锁存下来，下一拍再交给后一级使用。

本实验中的段寄存器直接体现在 `PipelineCPU.v` 的寄存器定义中。IF/ID 保存取指阶段得到的指令和 PC；ID/EX 保存译码结果、源操作数、立即数和控制信号；EX/MEM 保存执行结果、store 写数据和访存控制信号；MEM/WB 保存内存读数或 ALU 结果并在 WB 阶段写回寄存器。段寄存器的存在使每条指令在不同阶段之间携带自己的数据和控制信号，避免后续指令覆盖前一条指令尚未使用的信息。也就是说，单周期 CPU 的控制信号只在当前周期使用，而流水线 CPU 的控制信号必须随着指令一路向后传递。

Listing 4: 流水线 CPU 中四组段寄存器

```

1  reg          ifid_valid;
2  reg [31:0] ifid_pc;
3  reg [31:0] ifid_inst;

```

```
4
5 reg      idex_valid;
6 reg [31:0] idex_pc;
7 reg [31:0] idex_rd1;
8 reg [31:0] idex_rd2;
9 reg [31:0] idex_imm;
10 reg [4:0] idex_rs1;
11 reg [4:0] idex_rs2;
12 reg [4:0] idex_rd;
13 reg [4:0] idex_alu_op;
14 reg      idex_reg_write;
15 reg      idex_mem_read;
16 reg      idex_mem_write;
17 reg      idex_alu_src;
18 reg [1:0] idex_wb_sel;
19 reg      idex_branch;
20 reg      idex_jal;
21 reg      idex_jalr;
22
23 reg      exmem_valid;
24 reg [31:0] exmem_alu_result;
25 reg [31:0] exmem_store_data;
26 reg [4:0] exmem_rd;
27
28 reg      memwb_valid;
29 reg [31:0] memwb_alu_result;
30 reg [31:0] memwb_mem_data;
31 reg [4:0] memwb_rd;
```

代码截图占位 C1-pipeline-registers: 请截取
source-pipeline/PipelineCPU.v 第 20–62 行 IF/ID、ID/EX、EX/MEM、
MEM/WB 四组段寄存器定义。建议图片命名为
C1-pipeline-registers.png。

图 1: 代码截图占位 C1-pipeline-registers: 请截取 source-pipeline/PipelineCPU.v 第 20–62 行 IF/ID、ID/EX、EX/MEM、MEM/WB 四组段寄存器定义。建议图片命名为 C1-pipeline-registers.png。

流水线译码逻辑被整理到 `pipe_decode.v`。该模块从指令字段中生成 `rs1`、`rs2`、`rd`、立即数、ALU 操作码、写回选择和分支跳转控制信号。`PipelineCPU.v` 在 ID 阶段实例化该模块，并在时钟上升沿把这些控制信号写入 ID/EX 段寄存器。ALU 和分支判断也分别由 `pipe_alu.v` 与 `pipe_branch.v` 完成，使顶层代码集中在流水线寄存器推进和模块连接上。这样的模块划分也使流水线与单周期 CPU 的差异更清晰：单周期 CPU 的译码、ALU 和 NPC 逻辑在一个周期内直接组合连接，流水线 CPU 则需要把 ID 阶段产生的控制信号保存到 ID/EX，再由 EX 阶段模块使用。

Listing 5: 流水线 CPU 顶层对子模块的连接

```

1  pipe_decode U_DECODE(
2      .inst(ifid_inst),
3      .rs1(id_rs1), .rs2(id_rs2), .rd(id_rd),
4      .imm(id_imm), .alu_op(id_alu_op),
5      .reg_write(id_reg_write),
6      .mem_read(id_mem_read), .mem_write(id_mem_write),
7      .alu_src(id_alu_src), .wb_sel(id_wb_sel),
8      .branch(id_branch), .jal(id_jal), .jalr(id_jalr),
9      .uses_rs2(id_uses_rs2)
10 );
11
12 pipe_alu U_ALU(
13     .a(alu_in_a),
14     .b(alu_in_b),
15     .alu_op(idex_alu_op),
16     .result(alu_result)
17 );
18
19 pipe_branch U_BRANCH(
20     .valid(idex_valid),
21     .branch(idex_branch),
22     .jal(idex_jal),
23     .jalr(idex_jalr),
24     .funct3(idex_funct3),
25     .src_a(alu_in_a),
26     .src_b(rs2_forwarded),
27     .pc(idex_pc),
28     .imm(idex_imm),
29     .take_branch(ex_take_branch),
30     .branch_target(ex_branch_target),

```

```

31     .pc4_result(ex_pc4_result)
32 );

```

代码截图占位 C0-pipeline-modules: 请截取
source-pipeline/PipelineCPU.v 第 102–177 行 pipe_decode、pipe_alu、
pipe_branch 三个模块实例化代码。建议图片命名为
C0-pipeline-modules.png。

图 2: 代码截图占位 C0-pipeline-modules: 请截取 source-pipeline/PipelineCPU.v 第 102–177 行 pipe_decode、pipe_alu、pipe_branch 三个模块实例化代码。建议图片命名为 C0-pipeline-modules.png。

普通 RAW 数据冒险通过转发解决。RAW 是 read after write, 即后一条指令要读取的寄存器正好是前一条或前两条指令要写回的寄存器。例如 add x1,x2,x3 后紧跟 sub x4,x1,x5 时, sub 在 EX 阶段需要 x1 的新值, 但该值可能还没有写回寄存器堆。流水线 CPU 因此不能只依赖 RF 读口, 而要在 EX 阶段用 forward_a 和 forward_b 选择 ALU 输入。当结果已经在 EX/MEM 阶段可见时, 优先从 exmem_alu_result 转发; 当结果已经进入 MEM/WB 阶段时, 从 wb_data 转发。hazard_unit.v 对 EX/MEM 的转发额外排除了 mem_read, 因为 load 指令的数据在 EX/MEM 阶段还没有从数据存储返回, 不能作为有效转发源。

Listing 6: 流水线 CPU 的数据转发选择

```

1  assign forward_a =
2      (exmem_reg_write && !exmem_mem_read && (exmem_rd != 5'd0) && (exmem_rd
      == idex_rs1)) ? 2'b10 :
3      (memwb_reg_write && (memwb_rd != 5'd0) && (memwb_rd == idex_rs1)) ? 2'
      b01 :
4      2'b00;
5
6  assign forward_b =
7      (exmem_reg_write && !exmem_mem_read && (exmem_rd != 5'd0) && (exmem_rd
      == idex_rs2)) ? 2'b10 :
8      (memwb_reg_write && (memwb_rd != 5'd0) && (memwb_rd == idex_rs2)) ? 2'
      b01 :
9      2'b00;

```

代码截图占位 C2-forwarding: 请截取 source-pipeline/hazard_unit.v 第 35–43 行 forward_a 和 forward_b 的数据转发判断。建议图片命名为 C2-forwarding.png。

图 3: 代码截图占位 C2-forwarding: 请截取 source-pipeline/hazard_unit.v 第 35–43 行 forward_a 和 forward_b 的数据转发判断。建议图片命名为 C2-forwarding.png。

load-use 冒险不能只靠转发解决, 因为 lw 的数据要到 MEM 阶段结束后才可用, 而紧随其后的相关指令在下一拍就会进入 EX 阶段。假设指令序列为 lw x1,0(x2) 后接 add x3,x1,x4, 当 add 进入 EX 阶段时, lw 正在 MEM 阶段等待数据存储器输出, EX/MEM 中没有可转发的最终 load 数据。实验中通过 load_use_hazard 检测 ID 阶段源寄存器是否依赖 ID/EX 阶段的 load 目的寄存器。检测成立时, PC 和 IF/ID 暂停一拍, ID/EX 被冲成 bubble, 使 load 有时间进入 MEM/WB, 下一拍再通过 MEM/WB 转发给后续指令。

Listing 7: load-use 暂停与 bubble 插入

```
1 assign load_use_hazard =  
2     idex_mem_read &&  
3     (idex_rd != 5'd0) &&  
4     ((idex_rd == id_rs1) || (id_uses_rs2 && (idex_rd == id_rs2)));  
5  
6 assign pc_write    = ~load_use_hazard;  
7 assign ifid_write  = ~load_use_hazard;  
8 assign idex_flush  = load_use_hazard | ex_take_branch;  
9 assign ifid_flush  = ex_take_branch;
```

代码截图占位 C3-load-use-stall: 请截取 source-pipeline/hazard_unit.v 第 25–33 行 load_use_hazard、pc_write、ifid_write、idex_flush 的代码。建议图片命名为 C3-load-use-stall.png。

图 4: 代码截图占位 C3-load-use-stall: 请截取 source-pipeline/hazard_unit.v 第 25–33 行 load_use_hazard、pc_write、ifid_write、idex_flush 的代码。建议图片命名为 C3-load-use-stall.png。

控制冒险由 pipe_branch.v 和 hazard_unit.v 配合处理。分支、jal 和 jalr 的跳转目标在 EX 阶段确定，在此之前 IF 阶段可能已经按 PC+4 取入了顺序路径上的指令。如果 EX 阶段确认要跳转，ex_take_branch 置 1，PC 在时序逻辑中更新为 ex_branch_target，同时 IF/ID 和 ID/EX 被清空。这样已经被顺序取入的错误路径指令不会继续向后执行，也不会产生错误写回或错误访存。单周期 CPU 不存在这个问题，因为单周期 CPU 在同一周期内就能用当前指令决定下一 PC，没有后续指令提前进入流水线。

Listing 8: 跳转目标计算与流水线冲刷

```

1  assign take_branch = valid && ((branch && branch_ok) || jal || jalr);
2  assign branch_target = jalr ? ((src_a + imm) & 32'hffff_fffe) : (pc + imm)
   ;
3  assign pc4_result = pc + 32'd4;
4
5  if (ex_take_branch) begin
6      pc <= ex_branch_target;
7  end else if (pc_write) begin
8      pc <= pc + 32'd4;
9  end
10
11 if (ifid_flush) begin
12     ifid_valid <= 1'b0;
13     ifid_pc <= 32'b0;
14     ifid_inst <= 32'b0;
15 end

```

代码截图占位 B2-jalr-branch-flush: 请截取
source-pipeline/pipe_branch.v 第 29–31 行
take_branch/branch_target, 以及 source-pipeline/PipelineCPU.v 第
308–317 行 PC 更新和 IF/ID flush。建议图片命名为
B2-jalr-branch-flush.png。

图 5: 代码截图占位 B2-jalr-branch-flush: 请截取 source-pipeline/pipe_branch.v 第 29–31 行 take_branch/branch_target, 以及 source-pipeline/PipelineCPU.v 第 308–317 行 PC 更新和 IF/ID flush。建议图片命名为 B2-jalr-branch-flush.png。

6 两条指令的代码级执行过程

6.1 流水线 CPU 中运算指令 add 的执行过程

以 add x3, x1, x2 为例, 流水线 CPU 中该指令不会在一个周期内完成全部动作, 而是依次经过 IF、ID、EX、MEM、WB 五个阶段。它的语义仍然是读取 x1 和 x2, 把二者相加后的结果写入 x3; 但与单周期 CPU 不同, 流水线实现必须把每一阶段产生的数据和控制信号写入段寄存器, 使下一拍的后续阶段能够继续使用这些信息。

第一步, IF 阶段由 PipelineCPU.v 输出当前 pc 作为 PC_out, 指令存储器在 plcomp.v 中根据 PC[31:2] 取出 instr, 再送回 PipelineCPU.v 的 inst_in。时钟上升沿到来时, 如果没有暂停或冲刷, IF/ID 段寄存器保存当前 PC 和当前指令。对 add x3,x1,x2 来说, 这一步把该指令放入 ifid_inst, 使下一拍 ID 阶段可以译码。

第二步, ID 阶段由 pipe_decode.v 对 ifid_inst 拆字段。R 型 add 的 opcode 是 0110011, funct7=0000000, funct3=000。译码模块在 R 型分支中设置 reg_write=1、uses_rs2=1, 并把 alu_op 置为 PIPE_ALU_ADD。与此同时, 寄存器堆根据 ifid_inst[19:15] 和 ifid_inst[24:20] 读出 x1 和 x2。时钟上升沿到来时, ID/EX 段寄存器保存 rf_rd1、rf_rd2、id_rs1、id_rs2、id_rd 和 id_alu_op 等信息。

第三步, EX 阶段使用 forward_a 和 forward_b 先判断是否需要数据转发。若 x1 或 x2 正好来自前序尚未写回的指令, PipelineCPU.v 会从 EX/MEM 或 MEM/WB 选择最新值; 若没有相关冒险, 则直接使用 ID/EX 段寄存器中的 idex_rd1 和 idex_rd2。R 型 add 的 idex_alu_src=0, 所以 ALU 第二操作数选择 rs2_forwarded。随后 pipe_alu.v 在 PIPE_ALU_ADD 分支执行 a+b, 结果成为 alu_result。

第四步, EX/MEM 段寄存器保存 EX 阶段的结果。对 add 来说, ex_result 等于 alu_result, 随后进入 exmem_alu_result; 目的寄存器 x3 进入 exmem_rd; 写回控制 idex_reg_write 和 idex_wb_sel 也一起向后传递。第五步, MEM 阶段对 add 不访问数据存储器, 只是把 EX/MEM 结果继续写入 MEM/WB 段寄存器。第六步, WB 阶

段由 `wb_data` 选择 ALU 结果，并通过寄存器堆写端口把结果写回 `x3`。

Listing 9: 流水线 `add`: IF/ID 和 ID/EX 保存指令与译码结果

```

1  if (ifid_write) begin
2      ifid_valid <= 1'b1;
3      ifid_pc <= pc;
4      ifid_inst <= inst_in;
5  end
6
7  7'b0110011: begin
8      reg_write = 1'b1;
9      uses_rs2 = 1'b1;
10     case ({funct7, funct3})
11         {7'b0000000, 3'b000}: alu_op = `PIPE_ALU_ADD;
12         ...
13     endcase
14 end
15
16 idex_rd1 <= rf_rd1;
17 idex_rd2 <= rf_rd2;
18 idex_rs1 <= id_rs1;
19 idex_rs2 <= id_rs2;
20 idex_rd <= id_rd;
21 idex_alu_op <= id_alu_op;
22 idex_reg_write <= id_reg_write;

```

代码截图占位 A1-add-译码与段寄存器：请截取
`source-pipeline/pipe_decode.v` 第 57–72 行 R-type/add 译码，以及
`source-pipeline/PipelineCPU.v` 第 318–322 行 IF/ID 保存、第 287–305
 行 ID/EX 保存。建议图片命名为 `A1-add-decode-idex.png`。

图 6: 代码截图占位 A1-add-译码与段寄存器：请截取 `source-pipeline/pipe_decode.v` 第 57–72 行 R-type/add 译码，以及 `source-pipeline/PipelineCPU.v` 第 318–322 行 IF/ID 保存、第 287–305 行 ID/EX 保存。建议图片命名为 `A1-add-decode-idex.png`。

Listing 10: 流水线 `add`: 转发选择、ALU 加法和写回

```

1  case (forward_a)
2  2'b10: alu_in_a = exmem_alu_result;
3  2'b01: alu_in_a = wb_data;
4  default: alu_in_a = idex_rd1;
5  endcase
6
7  wire [31:0] alu_in_b = idex_alu_src ? idex_imm : rs2_forwarded;
8
9  `PIPE_ALU_ADD: result = a + b;
10
11 exmem_alu_result <= ex_result;
12 memwb_alu_result <= exmem_alu_result;
13
14 wire [31:0] wb_data =
15     (memwb_wb_sel == `PIPE_WB_MEM) ? memwb_mem_data : memwb_alu_result;

```

代码截图占位 A2-add-执行与写回：请截取
source-pipeline/PipelineCPU.v 第 136–156 行转发输入和 ALU 实例化、
第 250–264 行 EX/MEM 到 MEM/WB 传递，以及
source-pipeline/pipe_alu.v 第 10–14 行 PIPE_ALU_ADD。建议图片命名
为 A2-add-ex-wb.png。

图 7：代码截图占位 A2-add-执行与写回：请截取 source-pipeline/PipelineCPU.v 第 136–156 行转发输入和 ALU 实例化、第 250–264 行 EX/MEM 到 MEM/WB 传递，以及 source-pipeline/pipe_alu.v 第 10–14 行 PIPE_ALU_ADD。建议图片命名为 A2-add-ex-wb.png。

6.2 流水线 CPU 中跳转指令 jalr 的执行过程

以 jalr x0, x1, 0 为例,该指令常用于函数返回。它的目标地址是 $(x1+0) \& 32'hffff_fffe$, 并且理论上把 PC+4 写入 rd。本例中 rd 为 x0, 因此最终不会改变寄存器 x0; 真正可见的效果是 PC 跳转到 x1 保存的地址。流水线 CPU 中演示 jalr 更能体现控制冒险, 因为跳转目标在 EX 阶段才确定, 前面已经按顺序路径取进来的指令必须被冲刷。

第一步, jalr 在 IF 阶段进入 IF/ID 段寄存器。第二步, ID 阶段由 pipe_decode.v 识别 opcode 1100111, 设置 imm=imm_i、jalr=1、alu_src=1、reg_write=1。这些控制信号随后进入 ID/EX 段寄存器, 其中 idex_jalr 用来告诉 EX 阶段当前指令是 jalr, idex_imm 保存 I 型立即数, idex_rd 保存目的寄存器 x0。

第三步, EX 阶段先通过 `forward_a` 选择 `x1` 的最新值。如果前一条指令刚写 `x1`, 目标地址不能使用寄存器堆旧值, 而要从 EX/MEM 或 MEM/WB 转发。第四步, `pipe_branch.v` 看到 `jalr=1` 后让 `take_branch` 成立, 并计算 `branch_target = (src_a + imm) & 32'hffff_ffff`。这就是 `jalr` 的跳转状态判断和目标地址实现。第五步, `PipelineCPU.v` 在时钟上升沿检查 `ex_take_branch`, 若成立则把 PC 更新为 `ex_branch_target`。同时 `hazard_unit.v` 输出 `ifid_flush=1` 和 `idex_flush=1`, 清掉错误路径指令, 防止顺序取入的指令继续写寄存器或写内存。

Listing 11: 流水线 `jalr`: 译码和 ID/EX 控制信号保存

```

1 7'b1100111: begin // jalr
2     imm      = imm_i;
3     jalr      = 1'b1;
4     alu_src   = 1'b1;
5     reg_write = 1'b1;
6 end
7
8 idex_imm <= id_imm;
9 idex_rs1 <= id_rs1;
10 idex_rd <= id_rd;
11 idex_reg_write <= id_reg_write;
12 idex_alu_src <= id_alu_src;
13 idex_jalr <= id_jalr;

```

代码截图占位 B1-jalr-译码与 ID/EX: 请截取
`source-pipeline/pipe_decode.v` 第 115–120 行 `jalr` 译码, 以及
`source-pipeline/PipelineCPU.v` 第 291–305 行 ID/EX 保存 `jalr` 控制信号。建议图片命名为 `B1-jalr-decode-idex.png`。

图 8: 代码截图占位 B1-jalr-译码与 ID/EX: 请截取 `source-pipeline/pipe_decode.v` 第 115–120 行 `jalr` 译码, 以及 `source-pipeline/PipelineCPU.v` 第 291–305 行 ID/EX 保存 `jalr` 控制信号。建议图片命名为 `B1-jalr-decode-idex.png`。

Listing 12: 流水线 `jalr`: 跳转判断、目标地址、PC 更新和冲刷

```

1 assign take_branch = valid && ((branch && branch_ok) || jal || jalr);
2 assign branch_target = jalr ? ((src_a + imm) & 32'hffff_ffff) : (pc + imm)
;

```

```

3
4  if (ex_take_branch) begin
5      pc <= ex_branch_target;
6  end
7
8  assign ifid_flush = ex_take_branch;
9  assign idex_flush = load_use_hazard | ex_take_branch;

```

代码截图占位 B2-jalr-跳转与冲刷：请截取
source-pipeline/pipe_branch.v 第 29–31 行
take_branch/branch_target/pc4_result,
source-pipeline/PipelineCPU.v 第 308–317 行 PC 更新和 IF/ID flush,
以及 source-pipeline/hazard_unit.v 第 30–33 行 flush 输出。建议图片
命名为 B2-jalr-branch-flush.png。

图 9: 代码截图占位 B2-jalr-跳转与冲刷：请截取 source-pipeline/pipe_branch.v 第 29–31 行 take_branch/branch_target/pc4_result, source-pipeline/PipelineCPU.v 第 308–317 行 PC 更新和 IF/ID flush, 以及 source-pipeline/hazard_unit.v 第 30–33 行 flush 输出。建议图片命名为 B2-jalr-branch-flush.png。

7 仿真结果

最终扩展指令测试脚本共运行 16 个测试点，其中单周期 CPU 7 个测试点，流水线 CPU 9 个测试点。测试覆盖 slt、sltu、andi、ori、xori、slli、srli、srai、slti、sltiu、条件分支以及 jal/jalr。本地运行结果为 16 个测试点全部通过，说明扩展指令在当前 testbench 和期望寄存器检查下行为一致。

Listing 13: 最终扩展指令自动测试输出

```

1  [PASS] sc/sc_compare_slt.dat
2  [PASS] sc/sc_compare_sltu.dat
3  [PASS] sc/sc_logic_immediate.dat
4  [PASS] sc/sc_shift_immediate.dat
5  [PASS] sc/sc_set_less_immediate.dat
6  [PASS] sc/sc_branches.dat
7  [PASS] sc/sc_jalr.dat
8  [PASS] pl/pl_compare_slt.dat
9  [PASS] pl/pl_compare_sltu.dat
10 [PASS] pl/pl_logic_immediate.dat

```

```
11 [PASS] pl/pl_shift_immediate.dat
12 [PASS] pl/pl_set_less_immediate.dat
13 [PASS] pl/pl_branches.dat
14 [PASS] pl/pl_beq.dat
15 [PASS] pl/pl_jal.dat
16 [PASS] pl/pl_jalr.dat
17 Summary: 16/16 passed
```

学号排序程序分别在单周期、多周期和流水线 CPU 上运行。三种实现均输出原始学号 54873530 和排序后结果 03345578。排序结果中的数字升序排列，且没有丢失原始数字，说明比较、条件跳转、访存和子过程返回等路径在该程序下能够协同工作。流水线版本得到相同结果，进一步说明转发、暂停和冲刷没有破坏排序循环中的数据依赖和控制流。

表 3: 学号排序仿真结果

CPU 类型	仿真命令入口	原始学号	排序结果
单周期 CPU	source-sc 的 make run	54873530	03345578
多周期 CPU	source-mc 的 make run	54873530	03345578
流水线 CPU	source-pipeline 的 make run	54873530	03345578

Listing 14: 三类 CPU 学号排序仿真关键输出

```
1 [PASS] lab-6 sid sorting simulation passed.
2 [PASS] lab-6 multi-cycle sid sorting simulation passed.
3 [PASS] lab-6 pipeline sid sorting simulation passed.
```

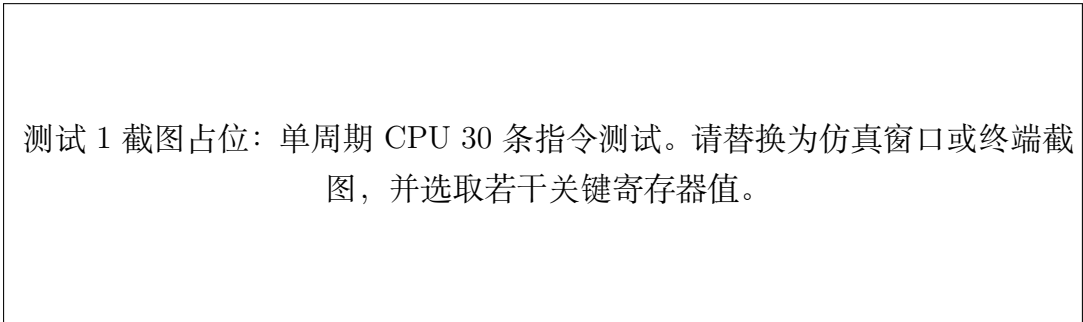


图 10: 测试 1 截图占位：单周期 CPU 30 条指令测试。请替换为仿真窗口或终端截图，并选取若干关键寄存器值。

测试 2 截图占位：流水线 CPU 30 条指令测试。请替换为仿真窗口或终端截图，并选取若干关键寄存器值。

图 11: 测试 2 截图占位：流水线 CPU 30 条指令测试。请替换为仿真窗口或终端截图，并选取若干关键寄存器值。

测试 3 截图占位：单周期 CPU 学号排序测试。建议展示 original_sid=54873530、sorted_sid=03345578 或对应寄存器/内存值。

图 12: 测试 3 截图占位：单周期 CPU 学号排序测试。建议展示 original_sid=54873530、sorted_sid=03345578 或对应寄存器/内存值。

测试 4 截图占位：流水线 CPU 学号排序测试。建议展示 original_sid=54873530、sorted_sid=03345578 或对应寄存器/内存值。

图 13: 测试 4 截图占位：流水线 CPU 学号排序测试。建议展示 original_sid=54873530、sorted_sid=03345578 或对应寄存器/内存值。

8 开发板运行结果

开发板演示部分采用 Nexys A7 相关封装文件。board_mem.v 提供上板使用的指令和数据存储接口，hex8_7seg.v 负责八位七段数码管显示，nexys_a7.xdc 提供约束文件。预期演示方式是复位后显示原始学号 54873530，按键触发排序运行后显示 03345578。当前报告保留开发板照片占位，实际提交前需要替换为板上运行照片或视频截图。

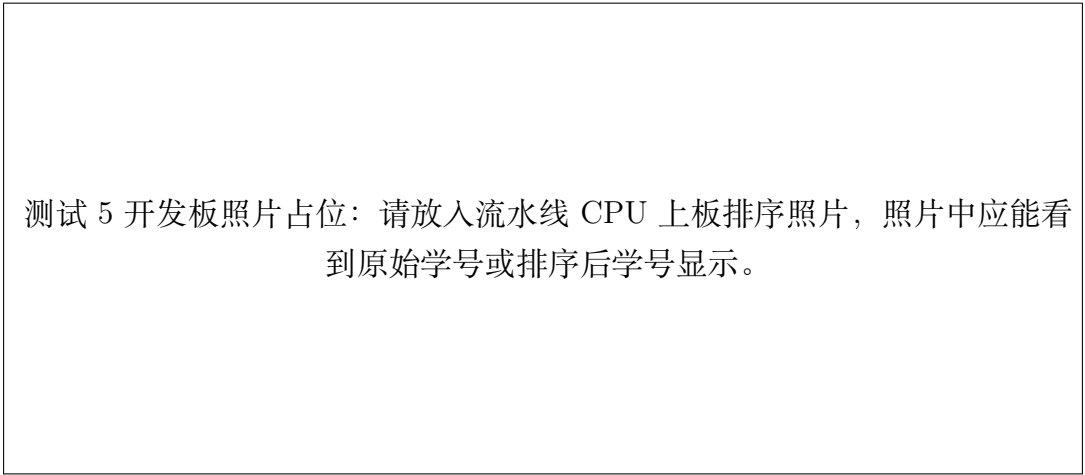


图 14: 测试 5 开发板照片占位：请放入流水线 CPU 上板排序照片，照片中应能看到原始学号或排序后学号显示。

9 代码与 diff 提交说明

实验代码需要上传到个人 GitHub 仓库，并在最终报告中给出仓库链接。当前报告先保留链接占位，正式提交前应替换为可访问的仓库地址。由于本地 lab-6 目录不是 git 仓库，报告不能从本地直接生成标准 `git diff --stat`；实际提交时应在 GitHub 或课程建议的 diff 网页中生成修改统计截图，并替换下面的占位图。为了说明代码规模，表 4 列出本次报告涉及的主要实现文件及当前行数。

表 4: 主要实现文件规模

文件	当前行数
source-sc/SCCPU.v	112
source-sc/ctrl.v	146
source-sc/alu.v	45
source-sc/NPC.v	23
source-mc/MCCPU.v	256
source-pipeline/PipelineCPU.v	325
source-pipeline/pipe_defs.v	19
source-pipeline/pipe_decode.v	125
source-pipeline/pipe_alu.v	26
source-pipeline/pipe_branch.v	32
source-pipeline/hazard_unit.v	44
final-验收文档/run_final_tests.py	90

GitHub 链接: <https://github.com/username/repository>

diff 网页顶部统计截图占位: 请替换为 GitHub diff 或课程工具生成的 diff 网页截图, 截图只需展示修改文件列表和每个文件增删行数。

图 15: diff 网页顶部统计截图占位: 请替换为 GitHub diff 或课程工具生成的 diff 网页截图, 截图只需展示修改文件列表和每个文件增删行数。

10 结果分析

扩展指令自动测试通过的结果说明, 单周期 CPU 和流水线 CPU 对最终验收要求的指令集合具有一致的寄存器可见行为。比较指令测试区分了有符号和无符号语义, 逻辑立即数和移位立即数测试覆盖了立即数扩展与 shamt 低 5 位选择, 分支与跳转测试则同时检查了跳转成立、跳转不成立、链接寄存器写回和错误路径指令不生效。由于这些测试点被拆成多个小程序, 单个指令族的错误不会被大程序掩盖, 测试结果对定位实现问题更直接。

学号排序程序提供了比单条指令测试更接近综合程序的验证。排序过程需要反复比较数据、根据条件分支决定是否交换、通过 load/store 访问数据存储器, 并通过跳转维持循环结构。三类 CPU 均得到 03345578, 说明扩展指令不只是在孤立测试中可用, 也能支撑包含循环和数据交换的程序执行。流水线 CPU 在相同程序上通过, 说明 RAW 相关、load-use 相关和跳转后的错误路径清理在该程序中没有产生可见错误。

单周期 CPU 的实现重点在于控制信号完整性, 同一条组合路径同时连接译码、执行、访存和写回。多周期 CPU 的实现重点在于状态机是否在正确周期保存中间结果并触发写操作, 它不需要处理流水线冒险, 因为没有多条指令重叠占用不同阶段。流水线 CPU 的实现复杂度主要来自指令重叠执行, 段寄存器让吞吐率具备提升空间, 同时也使旧值读取、load 数据未就绪和跳转目标延迟确定成为必须显式处理的问题。本实验通过转发、暂停和冲刷把这些问题分别约束在 EX 输入选择、PC/IFID 写使能以及流水线寄存器清空三个位置, 结构上便于在验收提问时逐项解释。

11 结论

本实验完成了单周期、多周期和流水线 RISC-V CPU 的扩展实现，并使用教师提供的最终验收 testbench 与学号排序程序进行了验证。单周期 CPU 完成了扩展指令的译码、ALU 运算、分支跳转和写回路径；多周期 CPU 完成了同一程序在状态机结构下的执行；流水线 CPU 在模块化整理后通过 IF/ID、ID/EX、EX/MEM、MEM/WB 四组段寄存器推进指令，并通过 hazard_unit.v、pipe_branch.v 和转发选择逻辑处理数据冒险与控制冒险。仿真结果显示最终扩展指令测试 16/16 通过，三类 CPU 的学号排序结果均为 03345578。提交前仍需补充真实测试截图、开发板照片、GitHub 链接和 diff 统计截图，以满足最终报告中的图像证据要求。